M6: у каждого слоя своя работа

Магазин уже покупает. Теперь проверяем, понятно ли устройство проекта другому человеку.

models · хранение

User, CartItem, Order.
Колонки, внешние ключи,
связи и ограничения БД.

schemas · контракт

CartAdd, OrderRead.
Что принять, что отклонить,
какие поля вернуть.

routers · HTTP

Метод, путь, Depends,
вызов сервиса и код ответа.
Связывает API с логикой.

services · правила

Владелец, цена, остаток,
создание заказа и транзакция.
Решает, допустима ли покупка.

↳ Отдельные папки полезны, когда по ним действительно разделена ответственность.

Шаг 1. Убрать расчёт из роутера

Если слои смешаны, переносим один сценарий небольшим изменением и сохраняем поведение.

Псевдокод · что нужно разделить — Учебный пример

```
def create_order(request):
    cart = SELECT корзины
    total = сумма цен
    INSERT заказ и позиции
    UPDATE остатки
    DELETE корзины
    COMMIT
```

Реальный роутер · сокращённый фрагмент — Учебный пример

```
@router.post("", status_code=201,
             response_model=OrderRead)
def checkout(user: CurrentUser,
             body: CheckoutRequest | None =
None):
    return orders.checkout(user.id)
```

Здесь HTTP и вся покупка находятся
в одной функции. Границу трудно увидеть.

↳ Переносим всю бизнес-операцию, сохраняя одного владельца commit/rollback.

Шаг 2. Сделать ошибки предсказуемыми

Сервис сообщает причину, обработчик превращает её в HTTP после отката.

Сокращённый фрагмент · два слоя — Учебный пример

```
# services/orders.py
raise ShopError(409, "insufficient_stock")

# обработчик после выхода из транзакции
return JSONResponse(
    status_code=error.status,
    content={"detail": error.reason})
```

Код	Что объясняем клиенту
401 / 403	Не вошёл / не хватает прав
404	Объект недоступен или отсутствует
409	Состояние не позволяет операцию
422	Неверные входные поля
503	/health: БД недоступна

↳ Не превращаем исключение в успешный ответ. При отказе изменения не должны сохраняться.

Шаг 3. Зафиксировать окружение

Другой человек получает имена настроек и версии пакетов, затем создаёт свои локальные значения.

Файл	Что в нём
requirements.txt	Полный стек с точными версиями
.env.example	Названия и учебный пример настроек
.env	Локальная БД и JWT_SECRET
core/config.py	Чтение и проверка настроек
.gitignore	Исключение .env, .venv и кэшей

В активном venv · из корня проекта — Учебный пример

```
python -m pip install -r requirements.txt
python -m pip check
python setup_jwt.py

# При использовании Git:
git check-ignore .env
git ls-files -- .env
```

↳ .gitignore не удаляет уже отслеживаемый файл. Проверяем комплект, который действительно передаём.

Шаг 4. Написать README для нового человека

Инструкция должна начинаться с получения проекта и заканчиваться работающим полным API.

Начало инструкции

Требования → корень команд → venv → зависимости → .env → ключ JWT.

Затем запуск и проверка

БД → миграция → seed → сервер → /health → /docs → покупка.

Финальные команды · после настройки — Учебный пример

```
docker compose up -d --wait
python -m alembic upgrade head
python -m app.seed
python -m uvicorn app.final_main:app
# http://127.0.0.1:8000/docs
```

↳ Команда запуска должна открывать весь магазин. app.main:app в эталоне остаётся только M0–M1.

Шаг 5. Проверить запуск в чистой копии

Новый venv и новая пустая БД исключают зависимость от прежних пакетов и таблиц.

Этап	Что выполнено
Окружение	Новая копия → venv → install → pip check
База	Новая пустая БД → upgrade head → seed дважды
Приложение	18 операций HTTP → регрессия 85 + 76

↳ Рабочая база и том сохранены. Проверка использует отдельные временные ресурсы и удаляет только их.

Шаг 6. Пройти POST /orders по файлам

Проследите один запрос от входа до ответа, называя реальные функции своего проекта.

1. Вход и пользователь

schemas/orders.py проверяет тело.
dependencies.py получает User по JWT.

2. HTTP → сервис

routers/orders.py вызывает
services/orders.py: checkout(user.id).

3. Транзакция и таблицы

unit_of_work.py открывает Session.
Сервис читает и меняет ORM-модели.

4. DTO → commit → ответ

OrderRead собирается в Session.
После успешного commit возвращаем 201.

↳ Можно показать файл, объяснить его роль и предсказать результат ошибки — значит путь запроса понятен.

Принимаем M6: код можно воспроизвести

Сопоставляем структуру, инструкцию и фактический запуск, затем закрываем замечания.

Критерий	Что предъявить
Слои	Путь POST /orders по файлам
Ошибки	Код, причина и неизменность БД
Настройки	Пример, локальные значения, версии
README	Команды без скрытых шагов
Чистый запуск	Отчёт из новой копии и новой БД

Факт проверки эталона

43 финальные проверки.
Регрессия M2–M3: 85.
Регрессия M4–M5: 76.

↳ Записываем: исправлено, осталось, не проверялось. Уверенное «всё хорошо» не заменяет эти факты.

18 Финальная проверка и сдача

Финал: показать весь путь покупателя

В Swagger пройдите те же действия и сохраните результаты. Переключатель показывает факты HTTP-прогона.

Шаг	HTTP	Проверенный результат
Вход	200	Аня подтверждена через /auth/me
Каталог	200	101: 7990.00 / stock 3; 201: 5990.00 / stock 5
Корзина	200	Две позиции, три пары, сумма 21970.00
Заказ	201	21970.00; stock 1 и 4; корзина пуста
История	200	Свой заказ виден; чужой ID возвращает 404

↳ Используем реальные id из ответов. Числа 21970.00 и остатки 1/4 предполагают исходный seed.

Проверить отказы и сохранность данных

Каждый сценарий отправлен финальному приложению по HTTP; снимки БД сравнивались до и после.

Сценарий	Код	После отказа
Без токена / нет роли	401 / 403	Без изменений
Чужая корзина / заказ	404	Без изменений
Ноль / цена<0 / пустой email	422	Без изменений
Нехватка второй позиции	409	Весь заказ отклонён

↳ 422 проверяет форму,403 — право,409 — состояние. Во всех случаях не должно быть частичной записи.

18 Финальная проверка и сдача

Сверить задание с реальным API

Считаем операции как пару «метод + путь». Один URL может обслуживать несколько методов.

Группа	Операций	Проверка
Каталог	6	Чтение и admin CRUD
Вход	3	Регистрация, login, /auth/me
Корзина	4	Чтение и изменения
Заказы	4	Покупка, история, статус
Health	1	Ответ с живой БД

↳ 17 операций карты задания + GET /health = 18. Наличие в OpenAPI сверено с фактическими вызовами.

18 Финальная проверка и сдача

CONCEPT.md: объяснить своими словами

Пять вопросов из задания. Для каждого приведите пример из своего работающего магазина.

1	**API и REST.**	Что делают группы маршрутов; чем отличаются GET и POST?
2	**База и схема.**	Какие семь таблиц связаны; почему stock хранится у размера?
3	**Оформление.**	Цена → проверки → записи → commit; что произойдёт при ошибке?
4	**Слои кода.**	Покажите один endpoint в schemas, routers, services и models.
5	**Аутентификация.**	Откуда берётся User; чем различаются 401 и 403?

↳ Шаблон помогает начать. Готовый текст нейросети не доказывает понимания автора проекта.

Подготовить комплект для проверки

Проверяющему нужны код, способ запуска, ваше объяснение и видимый результат.

Код и воспроизводимость

Репозиторий или архив проекта.
app, alembic, compose, requirements.
README и .env.example.

Доказательства работы

Результаты проверок.
Замечания и исправления ревью
после M2, M3 и M5.

Личный результат ученика

Заполненный CONCEPT.md.
Короткое видео или кадры Swagger.
Регистрация → корзина → заказ.

Проверьте содержимое: .env, .venv,
токены и кэши не входят в комплект.

↳ Передаём то, что можно открыть и проверить. Шаблон CONCEPT и план видео ещё нужно заполнить и выполнить.

18 Финальная проверка и сдача

Последняя проверка: объяснить магазин

Без кода восстановите путь запроса и предскажите, что останется в базе после ошибки.

Успешный путь

JWT → User → сервис → транзакция.
Цены БД → Order + позиции → stock
→ очистка → commit → 201.

Целостный отказ

409, нового заказа нет.
Первая позиция не списана.
Исходная корзина сохранена.

↳ Вы прошли M0–M6. Финальный результат — работающий проект, воспроизводимые проверки и собственное объяснение.